

Day 5 — Production-Style Vector Search Index (Embeddings + RAG Foundation)

Assumptions

This guide is written against the following fixed assumptions. If your environment differs, adjust the relevant `.env` values and commands accordingly — the code itself does not need to change.

1. **Corpus:** arXiv ML papers, downloaded programmatically via the `arxiv` Python library.
2. **Embedding model:** `text-embedding-004` (768 dimensions, L2-normalized output).
3. **Distance metric:** `DOT_PRODUCT_DISTANCE` (mathematically equivalent to cosine similarity when vectors are normalized).
4. **Shard size:** `SMALL` — appropriate for under 1M vectors, which comfortably covers an intern-scale corpus of 200+ papers.
5. **Error handling philosophy:** Lenient / graceful degradation. A failed PDF, a failed embedding batch, or a failed upsert batch is logged and skipped — it never crashes the whole run. The pipeline is designed to make partial progress under partial failure.
6. **Operating system:** macOS. All shell commands are bash-compatible for macOS (Terminal.app or iTerm2, default `bash/zsh` interop assumed).
7. **Virtual environment name:** `myenv` (explicitly, not `venv`, not `.venv`).
8. **Prior state:** You have already completed earlier GCP/Vertex AI setup tasks and have `gcloud` CLI installed and authenticated (`gcloud auth login` and `gcloud auth application-default login` both done).
9. **LLM layering:** Groq's `llama-3.3-70b-versatile` sits on top of retrieval purely as the answer-synthesis layer — it never touches the index, never sees unretrieved documents, and is not used for embeddings.

SECTION 1 — Project Architecture & Overview

1.1 System Design Narrative

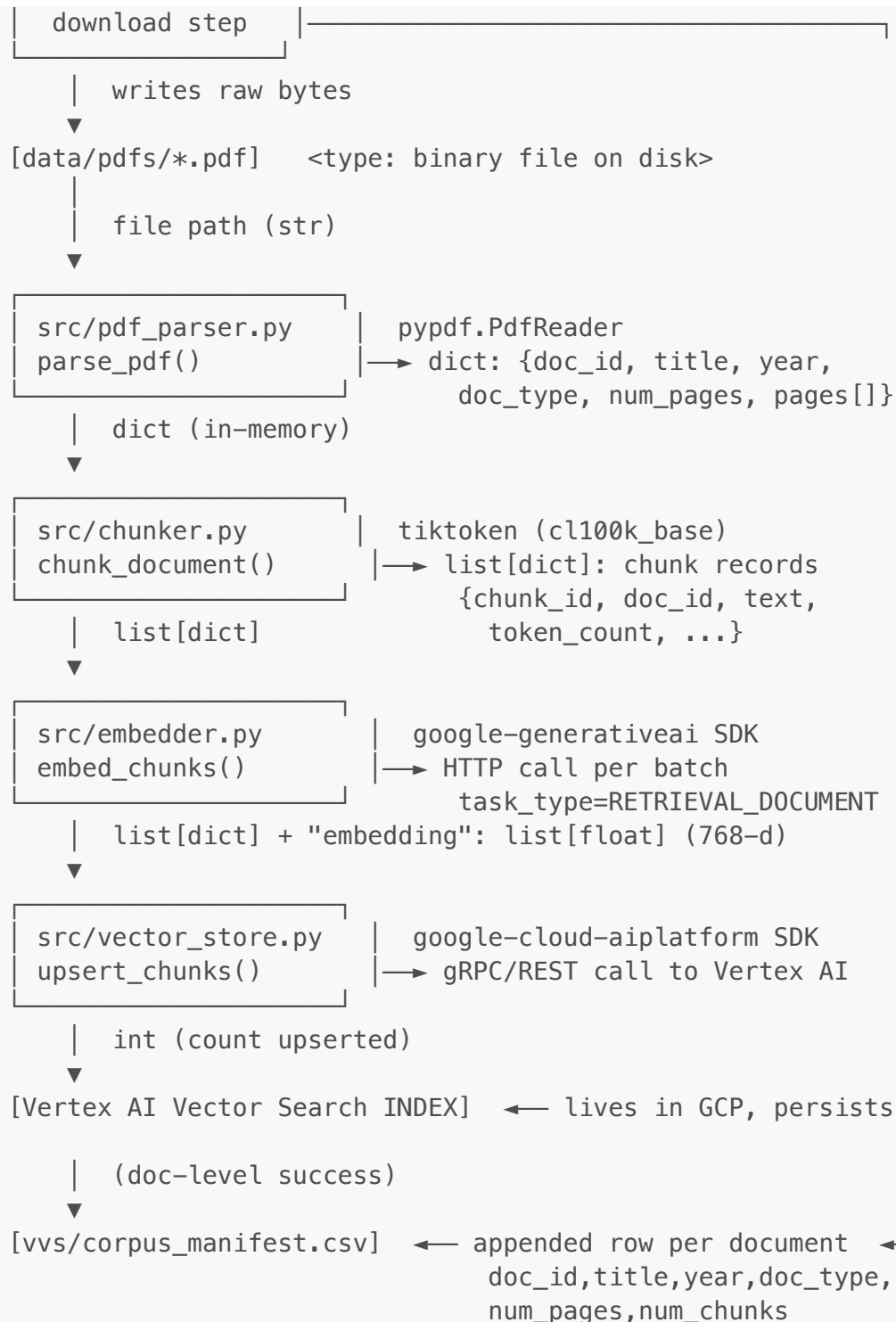
This project is a two-phase Retrieval-Augmented Generation (RAG) system. The two phases are deliberately decoupled — they run as separate scripts, on separate schedules, against separate concerns — because that separation is what makes the system production-shaped rather than notebook-shaped.

Phase 1 — Offline Indexing (`vvs/ingest.py`)

This phase runs once (or periodically, whenever the corpus grows). It is slow, API-call-heavy, and idempotent-ish by design (re-running it skips already-downloaded PDFs). Its job is to turn unstructured PDF bytes into searchable vectors sitting inside Vertex AI.

The pipeline, step by step:

1. **Download** — The `arxiv` library searches arXiv for papers matching a topic query (e.g. "transformer architecture") and downloads PDFs into `data/pdfs/`. Already-downloaded files are

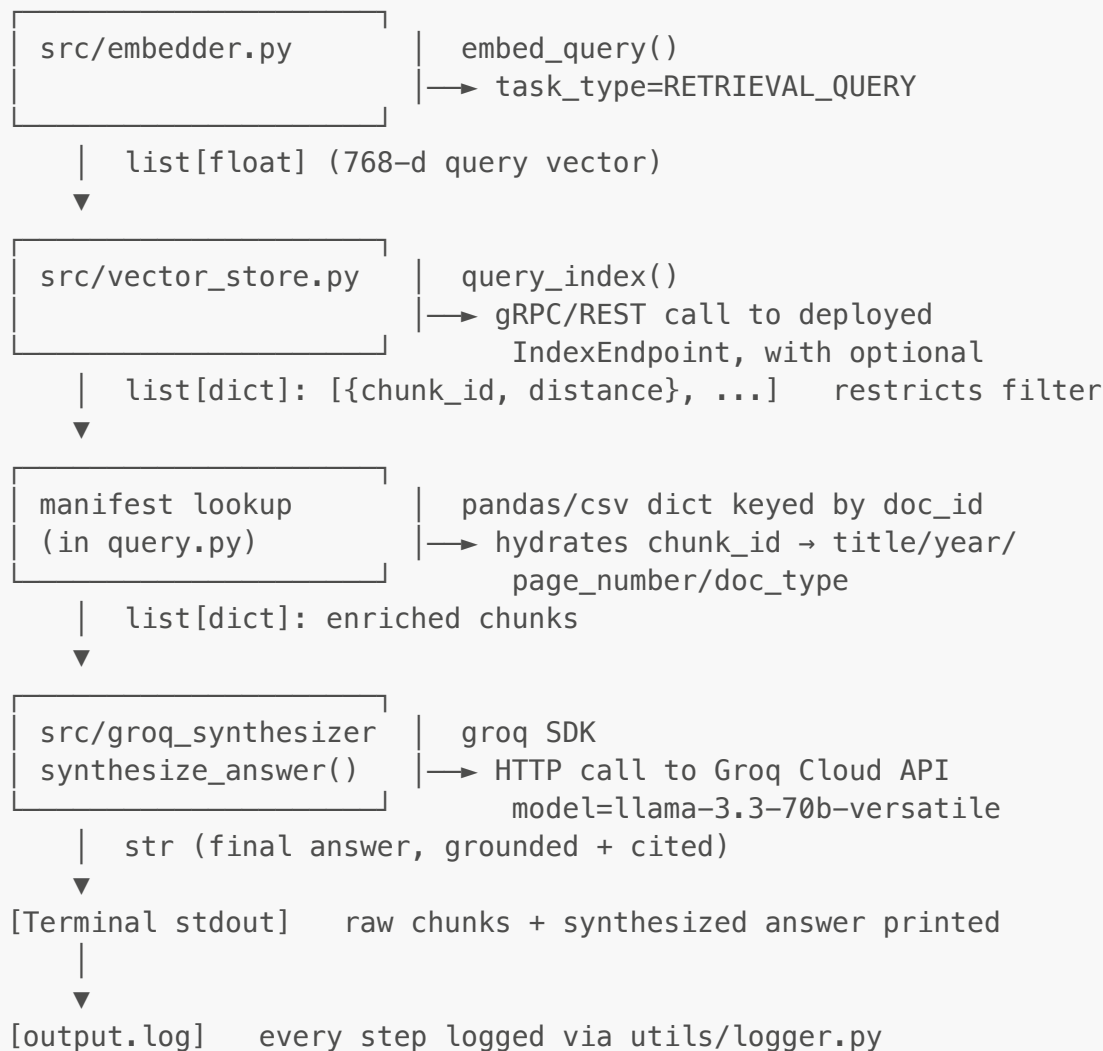


PHASE 2: ONLINE QUERY

[User CLI input] "--question '...' --year 2023"
| str (+ optional filters)

query.py | orchestrator

| question: str



1.3 Rationale for Every Technology Choice

Why `pypdf` over `pdfplumber` or `PyMuPDF`. `pypdf` is pure Python, has zero binary dependencies, and is trivially `pip install`-able on macOS without wrestling with compiled wheels (`PyMuPDF` ships a compiled `fitz` binding that occasionally has platform-specific install friction; `pdfplumber` itself depends on `pdfminer.six`, which is heavier and slower for the simple page-by-page text extraction this project needs). arXiv papers are typically well-formed, text-based PDFs (not scanned images), so the more sophisticated layout-reconstruction features of `pdfplumber` or the OCR-adjacent power of `PyMuPDF` are not needed here. `pypdf`'s page-by-page API is also the most direct match for this project's data model, where chunks need to know which page they came from.

Why `text-embedding-004` over `textembedding-gecko`. `text-embedding-004` is Google's newer-generation embedding model, accessible directly through the Gemini Embedding API (`google-generativeai`), and produces normalized 768-dimension vectors out of the box. `textembedding-gecko` is the older Vertex AI-native embedding model family; it requires going through the full Vertex AI Prediction API surface rather than the simpler Generative AI SDK, and its retrieval-quality benchmarks are behind `text-embedding-004`. Since this project already needs a Google AI Studio API key for simplicity (rather than provisioning another Vertex AI endpoint just for embeddings), `text-embedding-004` is the natural choice.

Why 512-token chunks with overlap. 512 tokens is large enough to hold a complete idea, a full paragraph, or several related sentences from a technical paper — small enough that a single chunk doesn't dilute the embedding with multiple unrelated topics (embeddings of very long, multi-topic text tend to become a "blurry average" that matches everything a little and nothing well). The overlap (64 tokens, 12.5% of chunk size) exists specifically so that an idea spanning a chunk boundary — e.g. a sentence that starts at token 500 and ends at token 530 — appears whole in at least one chunk rather than being severed in both.

Why Vertex AI Vector Search over Pinecone/Weaviate/pgvector. Three reasons specific to this project's stated constraints: (1) the project assumes prior GCP/Vertex AI setup is already done, so staying inside GCP avoids introducing a second cloud vendor and a second auth/billing surface; (2) Vertex AI Vector Search is a fully managed ANN service with no servers for an intern to operate, patch, or scale — pgvector, by contrast, requires you to run and tune a Postgres instance yourself; (3) Vertex AI's `restricts/namespace` filtering integrates metadata filtering directly into the ANN search at the index level, which is the feature this project's `--year/--doc_type` filters depend on.

Why DOT_PRODUCT_DISTANCE over COSINE or EUCLIDEAN. `text-embedding-004` already returns L2-normalized vectors (unit length). For normalized vectors, dot product and cosine similarity are mathematically identical (proven in Section 4.3) — but computing a raw dot product is cheaper than computing cosine similarity from scratch (which would otherwise require re-normalizing at query time). `EUCLIDEAN` distance is a fundamentally different geometric measure (straight-line distance) that is sensitive to vector magnitude — irrelevant and slightly misleading for embeddings whose semantic information is encoded in direction, not magnitude.

Why Groq's llama-3.3-70b-versatile for synthesis. Groq runs inference on custom LPU (Language Processing Unit) hardware, giving extremely low time-to-first-token and high tokens/second compared to typical GPU-served inference — important for a query pipeline where a user is waiting on an answer interactively. `llama-3.3-70b-versatile` has a context window large enough to comfortably hold 5–10 retrieved chunks of ~512 tokens each (2,500–5,000 tokens of context) plus the system and user prompt overhead, while being a strong-enough model to follow strict grounding instructions ("answer only from context") reliably.

Why arXiv as the corpus. arXiv papers are freely and legally redistributable for research and educational use, programmatically accessible via a stable, well-documented API (the `arxiv` Python library wraps arXiv's public Atom feed), available in large volume (200+ ML papers on a single topic is trivial to source), and structurally consistent (title, authors, abstract, PDF link are all present in the API response), making metadata extraction for the manifest straightforward and reliable.

SECTION 2 — Repository & Folder Structure

2.1 Project Tree

```
day5_vector_search/  
├── vvs/  
│   ├── ingest.py  
│   └── query.py
```

```

├── corpus_manifest.csv      # auto-generated, starts empty
├── filtered_queries.md     # filled after testing
├── setup.md                # Vertex AI config reference
├── src/
│   ├── __init__.py
│   ├── pdf_parser.py
│   ├── chunker.py
│   ├── embedder.py
│   ├── vector_store.py
│   └── groq_synthesizer.py
├── config/
│   ├── __init__.py
│   └── settings.py
├── utils/
│   ├── __init__.py
│   └── logger.py
├── tests/
│   ├── __init__.py
│   ├── test_chunker.py
│   ├── test_embedder.py
│   └── test_query.py
├── data/
│   └── pdfs/               # raw downloaded PDFs land here
├── logs/
│   └── .gitkeep
├── output.log              # auto-created by logger
├── .env
├── .gitignore
├── requirements.txt
├── setup.sh
└── README.md

```

2.2 Separation of Concerns

- **vvs/** — The "verbs" of the project. This is where the two primary deliverables live: **ingest.py** (the Phase 1 orchestrator) and **query.py** (the Phase 2 orchestrator). It also holds the artifacts those scripts produce or consume: **corpus_manifest.csv** (the source of truth for doc-level metadata), **filtered_queries.md** (a human-curated log of test queries and their results), and **setup.md** (a reference card for the Vertex AI resource IDs so they don't live only inside **.env**).
- **src/** — The "nouns" of the project: reusable, testable, side-effect-isolated modules. Each file owns exactly one external API boundary (**pdf_parser.py** owns **pypdf**, **embedder.py** owns the Gemini Embedding API, **vector_store.py** owns Vertex AI, **groq_synthesizer.py** owns Groq) or one pure-computation concern (**chunker.py** has no external API at all). This isolation is what makes the modules independently unit-testable — **test_chunker.py** can run with zero network access.
- **config/** — Centralizes every environment-variable read into a single module (**settings.py**) so that no other file in the project calls **os.getenv()** directly. This means there is exactly one place to look when a config value is wrong, and exactly one place that validates required keys are present at startup.

- **utils/** — Cross-cutting concerns used by every other module. Currently just `logger.py`, which exists here (not in `src/`) because logging isn't part of the RAG pipeline's domain logic — it's infrastructure that the domain logic depends on.
- **tests/** — Unit tests, mirroring the module names in `src/`. Kept flat (not mirroring the `src/config/` directory split) because the test count is small enough that a flat namespace is more discoverable than a deep one.
- **data/pdfs/** — The only directory where raw, untracked, potentially large binary files live. Isolated from `vvs/` and `src/` so that `.gitignore` can exclude an entire subtree (`data/pdfs/`) without risk of also excluding code.
- **logs/** — Reserved for future log rotation/archival (the active log is `output.log` at the project root for easy `tail -f` access during development); `.gitkeep` exists purely so the empty directory survives being committed to git.
- **Root-level files** (`output.log`, `.env`, `.gitignore`, `requirements.txt`, `setup.sh`, `README.md`) — Project-wide concerns that don't belong inside any single subpackage.

2.3 Scaffold Script

Copy-paste this entire block into a macOS Terminal to generate the full structure in one execution.

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="day5_vector_search"

echo "Creating project structure at ./${PROJECT_ROOT} ..."

mkdir -p "${PROJECT_ROOT}/vvs"
mkdir -p "${PROJECT_ROOT}/src"
mkdir -p "${PROJECT_ROOT}/config"
mkdir -p "${PROJECT_ROOT}/utils"
mkdir -p "${PROJECT_ROOT}/tests"
mkdir -p "${PROJECT_ROOT}/data/pdfs"
mkdir -p "${PROJECT_ROOT}/logs"

# vvs/
touch "${PROJECT_ROOT}/vvs/ingest.py"
touch "${PROJECT_ROOT}/vvs/query.py"
touch "${PROJECT_ROOT}/vvs/filtered_queries.md"
touch "${PROJECT_ROOT}/vvs/setup.md"

# src/
touch "${PROJECT_ROOT}/src/__init__.py"
touch "${PROJECT_ROOT}/src/pdf_parser.py"
touch "${PROJECT_ROOT}/src/chunker.py"
touch "${PROJECT_ROOT}/src/embedder.py"
touch "${PROJECT_ROOT}/src/vector_store.py"
touch "${PROJECT_ROOT}/src/groq_synthesizer.py"

# config/
touch "${PROJECT_ROOT}/config/__init__.py"
```

```
touch "${PROJECT_ROOT}/config/settings.py"

# utils/
touch "${PROJECT_ROOT}/utils/__init__.py"
touch "${PROJECT_ROOT}/utils/logger.py"

# tests/
touch "${PROJECT_ROOT}/tests/__init__.py"
touch "${PROJECT_ROOT}/tests/test_chunker.py"
touch "${PROJECT_ROOT}/tests/test_embedder.py"
touch "${PROJECT_ROOT}/tests/test_query.py"

# logs/
touch "${PROJECT_ROOT}/logs/.gitkeep"

# root files
touch "${PROJECT_ROOT}/output.log"
touch "${PROJECT_ROOT}/.env"

# corpus_manifest.csv with header row
cat > "${PROJECT_ROOT}/vvs/corpus_manifest.csv" << 'CSV_EOF'
doc_id,title,year,doc_type,num_pages,num_chunks
CSV_EOF

# .gitignore
cat > "${PROJECT_ROOT}/.gitignore" << 'GITIGNORE_EOF'
myenv/
.env
__pycache__/
*.pyc
output.log
data/pdfs/
logs/
GITIGNORE_EOF

# requirements.txt (pinned versions)
cat > "${PROJECT_ROOT}/requirements.txt" << 'REQ_EOF'
python-dotenv==1.0.1
pypdf==4.3.1
tiktoken==0.7.0
google-generativeai==0.7.2
google-cloud-aiplatform==1.62.0
groq==0.9.0
arxiv==2.1.3
pandas==2.2.2
REQ_EOF

# empty README
touch "${PROJECT_ROOT}/README.md"

echo "Project structure created successfully under ./${PROJECT_ROOT}"
echo "Next: cd ${PROJECT_ROOT} && bash setup.sh"
```

SECTION 3 — Production-Ready Implementation Code

3.1 `utils/logger.py`

```

"""
Singleton logger factory for the entire project.

Every module in this codebase calls get_logger() instead of using print().
This guarantees a single, consistently formatted log stream that goes to
both the terminal (for live feedback) and output.log (for post-hoc
debugging
and audit trails of long-running ingestion jobs).
"""

import logging
import os
import sys

_LOGGER_NAME = "vvs_logger"
_LOG_FILE_PATH =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"output.log")
_LOG_FORMAT = "%(asctime)s | %(levelname)s | %(message)s"
_DATE_FORMAT = "%Y-%m-%d %H:%M:%S"

_logger_instance = None

def get_logger() -> logging.Logger:
    """
    Returns the singleton 'vvs_logger' instance, configuring it on first
    call.

    Log level is INFO by default, or DEBUG if the LOG_LEVEL environment
    variable is set to "DEBUG" (case-insensitive).

    Returns:
        logging.Logger: the configured singleton logger.
    """
    global _logger_instance

    if _logger_instance is not None:
        return _logger_instance

    logger = logging.getLogger(_LOGGER_NAME)

    log_level_env = os.environ.get("LOG_LEVEL", "INFO").strip().upper()
    log_level = logging.DEBUG if log_level_env == "DEBUG" else logging.INFO
    logger.setLevel(log_level)

    # Prevent duplicate handlers if get_logger() is somehow called multiple

```

```

# times during a single process lifetime (e.g. reimport in tests).
if logger.handlers:
    _logger_instance = logger
    return _logger_instance

formatter = logging.Formatter(fmt=_LOG_FORMAT, datefmt=_DATE_FORMAT)

stream_handler = logging.StreamHandler(stream=sys.stdout)
stream_handler.setLevel(log_level)
stream_handler.setFormatter(formatter)
logger.addHandler(stream_handler)

file_handler = logging.FileHandler(_LOG_FILE_PATH, mode="a",
encoding="utf-8")
file_handler.setLevel(log_level)
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

logger.propagate = False

_logger_instance = logger
return _logger_instance

```

3.2 config/settings.py

```

"""
Centralized configuration loader.

Every other module in this codebase imports config values from here.
No other module should call os.getenv() directly – this is the single
source of truth for environment configuration, and the single place
that validates required keys are present at startup.
"""

import os

from dotenv import load_dotenv

load_dotenv()

GROQ_API_KEY = os.environ.get("GROQ_API_KEY")
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
GCP_PROJECT_ID = os.environ.get("GCP_PROJECT_ID")
GCP_REGION = os.environ.get("GCP_REGION", "us-central1")
VERTEX_INDEX_ID = os.environ.get("VERTEX_INDEX_ID")
VERTEX_INDEX_ENDPOINT_ID = os.environ.get("VERTEX_INDEX_ENDPOINT_ID")
VERTEX_DEPLOYED_INDEX_ID = os.environ.get("VERTEX_DEPLOYED_INDEX_ID")

EMBEDDING_MODEL = os.environ.get("EMBEDDING_MODEL", "models/text-embedding-
004")
EMBEDDING_DIMENSION = int(os.environ.get("EMBEDDING_DIMENSION", "768"))

```

```

CHUNK_SIZE_TOKENS = int(os.environ.get("CHUNK_SIZE_TOKENS", "512"))
CHUNK_OVERLAP_TOKENS = int(os.environ.get("CHUNK_OVERLAP_TOKENS", "64"))
TOP_K_RESULTS = int(os.environ.get("TOP_K_RESULTS", "5"))

PDF_DATA_DIR = os.environ.get("PDF_DATA_DIR", "data/pdfs")
MANIFEST_PATH = os.environ.get("MANIFEST_PATH", "vvs/corpus_manifest.csv")

_REQUIRED_KEYS = {
    "GROQ_API_KEY": GROQ_API_KEY,
    "GEMINI_API_KEY": GEMINI_API_KEY,
    "GCP_PROJECT_ID": GCP_PROJECT_ID,
    "VERTEX_INDEX_ID": VERTEX_INDEX_ID,
    "VERTEX_INDEX_ENDPOINT_ID": VERTEX_INDEX_ENDPOINT_ID,
    "VERTEX_DEPLOYED_INDEX_ID": VERTEX_DEPLOYED_INDEX_ID,
}

def validate_settings() -> None:
    """
    Validates that all critical configuration keys are present.

    Raises:
        EnvironmentError: if one or more required keys are missing, with a
            message listing exactly which keys are absent.
    """
    missing = [key for key, value in _REQUIRED_KEYS.items() if not value]
    if missing:
        raise EnvironmentError(
            "Missing required environment variable(s): "
            + ", ".join(missing)
            + ". Check your .env file against .env.example / Section 5.3 of
guide.md."
        )

```

3.3 src/pdf_parser.py

```

"""
PDF parsing module. Owns the pypdf API boundary exclusively – no other
module should import pypdf directly.
"""

import hashlib
import os
import re
import unicodedata

import pypdf
from pypdf.errors import PdfReadError

from utils.logger import get_logger

```

```

logger = get_logger()

_MIN_WORDS_PER_LINE = 3

def clean_text(text: str) -> str:
    """
    Cleans raw extracted PDF text.

    - Strips excessive whitespace (collapses runs of whitespace to a single
    space)
    - Removes null bytes
    - Normalizes unicode to NFKC form
    - Removes lines with fewer than 3 words (common PDF layout noise: page
    numbers, running headers, isolated figure captions, etc.)

    Args:
        text: raw text extracted from a PDF page.

    Returns:
        str: cleaned text.
    """
    if not text:
        return ""

    text = text.replace("\x00", "")
    text = unicodedata.normalize("NFKC", text)

    cleaned_lines = []
    for line in text.split("\n"):
        collapsed = re.sub(r"\s+", " ", line).strip()
        if not collapsed:
            continue
        word_count = len(collapsed.split(" "))
        if word_count < _MIN_WORDS_PER_LINE:
            continue
        cleaned_lines.append(collapsed)

    return "\n".join(cleaned_lines)

def _compute_doc_id(pdf_path: str) -> str:
    """Computes a SHA256 hash of file content and returns the first 12 hex
    chars."""
    sha256 = hashlib.sha256()
    with open(pdf_path, "rb") as f:
        for block in iter(lambda: f.read(65536), b''):
            sha256.update(block)
    return sha256.hexdigest()[:12]

def _extract_title(reader: "pypdf.PdfReader", pdf_path: str) -> str:
    """Extracts title from PDF metadata, falling back to filename."""
    try:

```

```

        metadata = reader.metadata
        if metadata is not None and metadata.title:
            title = str(metadata.title).strip()
            if title:
                return title
    except Exception:
        pass
    return os.path.splitext(os.path.basename(pdf_path))[0]

def _extract_year(reader: "pypdf.PdfReader") -> int:
    """Extracts publication/creation year from PDF metadata, falling back
    to 0."""
    try:
        metadata = reader.metadata
        if metadata is None:
            return 0
        raw_date = metadata.get("/CreationDate") or
metadata.get("/ModDate")
        if not raw_date:
            return 0
        # PDF date format: D:YYYYMMDDHHmmSS...
        match = re.search(r"D:(\d{4})", str(raw_date))
        if match:
            return int(match.group(1))
        return 0
    except Exception:
        return 0

def parse_pdf(pdf_path: str) -> dict:
    """
    Parses a single PDF file into a structured document dict.

    Args:
        pdf_path: filesystem path to the PDF file.

    Returns:
        dict with keys {doc_id, title, year, doc_type, num_pages, pages},
        or None if the file could not be parsed (error is logged).
    """
    logger.info(f"Parsing PDF: {pdf_path}")

    try:
        doc_id = _compute_doc_id(pdf_path)
    except (OSError, IOError) as e:
        logger.error(f"Failed to read file for hashing: {pdf_path} | {e}")
        return None

    try:
        reader = pypdf.PdfReader(pdf_path)
    except PdfReadError as e:
        logger.error(f"pypdf failed to read PDF (corrupt or encrypted):
{pdf_path} | {e}")

```

```

        return None
    except (OSError, IOError) as e:
        logger.error(f"File I/O error opening PDF: {pdf_path} | {e}")
        return None

    num_pages = len(reader.pages)
    logger.info(f"Found {num_pages} pages in {pdf_path} | doc_id={doc_id}")

    pages = []
    for page_number, page in enumerate(reader.pages, start=1):
        try:
            raw_text = page.extract_text() or ""
        except Exception as e:
            logger.error(f"Failed to extract text from page {page_number}
of {pdf_path} | {e}")
            raw_text = ""
        pages.append({
            "page_number": page_number,
            "text": clean_text(raw_text),
        })

    title = _extract_title(reader, pdf_path)
    year = _extract_year(reader)

    return {
        "doc_id": doc_id,
        "title": title,
        "year": year,
        "doc_type": "arxiv_paper",
        "num_pages": num_pages,
        "pages": pages,
    }

```

3.4 src/chunker.py

```

"""
Token-aware chunking module. Pure computation – no external API calls,
which makes this module trivially unit-testable.
"""

import tiktoken

from utils.logger import get_logger

logger = get_logger()

_ENCODING_NAME = "cl100k_base"
_MIN_CHUNK_TOKENS = 20

_encoding = tiktoken.get_encoding(_ENCODING_NAME)

```

```
def _join_pages(pages: list) -> str:
    """Concatenates page texts with a page-boundary separator."""
    parts = []
    for page in pages:
        parts.append(f"\n\n--- Page {page['page_number']} ---\n\n{page['text']}")
    return "".join(parts)

def _page_number_for_token_offset(pages: list, full_text: str, char_offset:
int) -> int:
    """
    Given a character offset into the joined full_text, determines which
    page that offset falls on by finding the nearest preceding page marker.
    Falls back to page 1 if no marker precedes the offset.
    """
    preceding_text = full_text[:char_offset]
    page_markers = list(_PAGE_MARKER_PATTERN.finditer(preceding_text))
    if not page_markers:
        return pages[0]["page_number"] if pages else 1
    return int(page_markers[-1].group(1))

import re # noqa: E402 (kept near point of use for clarity of the helper
above)

_PAGE_MARKER_PATTERN = re.compile(r"--- Page (\d+) ---")

def chunk_document(doc: dict, chunk_size: int, overlap: int) -> list:
    """
    Splits a parsed document into overlapping, token-bounded chunks using a
    sliding window algorithm.

    Args:
        doc: a document dict as returned by pdf_parser.parse_pdf().
        chunk_size: maximum tokens per chunk.
        overlap: number of tokens each successive chunk overlaps with the
        previous one.

    Returns:
        list[dict]: chunk records, each with keys
            {chunk_id, doc_id, title, year, doc_type, page_number,
             chunk_index, text, token_count}
    """
    doc_id = doc["doc_id"]
    pages = doc.get("pages", [])

    full_text = _join_pages(pages)
    all_token_ids = _encoding.encode(full_text)
    total_tokens = len(all_token_ids)

    if total_tokens == 0:
```

```

        logger.info(f"doc_id={doc_id} produced zero tokens after joining
pages – no chunks created.")
        return []

    stride = chunk_size - overlap
    if stride <= 0:
        raise ValueError(f"chunk_size ({chunk_size}) must be greater than
overlap ({overlap}).")

    chunks = []
    chunk_index = 0
    token_counts_for_log = []

    start = 0
    while start < total_tokens:
        end = min(start + chunk_size, total_tokens)
        token_window = all_token_ids[start:end]

        if len(token_window) < _MIN_CHUNK_TOKENS:
            # Degenerate trailing chunk – skip rather than index near-empty
content.
            break

        chunk_text = _encoding.decode(token_window)

        # Determine the char offset of this window's start, to attribute a
page number.
        prefix_text = _encoding.decode(all_token_ids[:start])
        page_number = _page_number_for_token_offset(pages, full_text,
len(prefix_text))

        chunk_record = {
            "chunk_id": f"{doc_id}_chunk_{chunk_index:04d}",
            "doc_id": doc_id,
            "title": doc["title"],
            "year": doc["year"],
            "doc_type": doc["doc_type"],
            "page_number": page_number,
            "chunk_index": chunk_index,
            "text": chunk_text,
            "token_count": len(token_window),
        }
        chunks.append(chunk_record)
        token_counts_for_log.append(len(token_window))

        chunk_index += 1

    if end == total_tokens:
        break
    start += stride

if token_counts_for_log:
    logger.info(
        f"doc_id={doc_id} | chunks_produced={len(chunks)} | "

```

```

        f"min_tokens={min(token_counts_for_log)} | "
        f"max_tokens={max(token_counts_for_log)} | "
        f"avg_tokens={sum(token_counts_for_log) /
len(token_counts_for_log):.1f}"
    )
    else:
        logger.info(f"doc_id={doc_id} | chunks_produced=0 (all windows
below minimum token threshold)")

    return chunks

```

3.5 src/embedder.py

```

"""
Embedding generation module. Owns the google-generativeai API boundary
exclusively – no other module should call genai.embed_content() directly.
"""

import time

import google.generativeai as genai
from google.api_core.exceptions import ResourceExhausted

from config.settings import GEMINI_API_KEY
from utils.logger import get_logger

logger = get_logger()

genai.configure(api_key=GEMINI_API_KEY)

_MAX_RETRIES = 5
_BACKOFF_DELAYS_SECONDS = [2, 4, 8, 16, 32]
_INTER_BATCH_SLEEP_SECONDS = 1

def _embed_with_retry(text: str, model: str, task_type: str) -> list:
    """
    Calls genai.embed_content() with exponential backoff retry on
    ResourceExhausted (rate limit / quota) errors.

    Args:
        text: text to embed.
        model: embedding model identifier.
        task_type: "RETRIEVAL_DOCUMENT" or "RETRIEVAL_QUERY".

    Returns:
        list[float]: the embedding vector.

    Raises:
        ResourceExhausted: if all retries are exhausted.
    """

```

```

        Exception: any other API exception, re-raised after logging.
    """
    last_exception = None
    for attempt in range(_MAX_RETRIES + 1):
        try:
            response = genai.embed_content(model=model, content=text,
            task_type=task_type)
            return response["embedding"]
        except ResourceExhausted as e:
            last_exception = e
            if attempt < _MAX_RETRIES:
                delay = _BACKOFF_DELAYS_SECONDS[attempt]
                logger.info(
                    f"ResourceExhausted on embed_content (attempt {attempt
+ 1}/{_MAX_RETRIES}). "
                    f"Retrying in {delay}s..."
                )
                time.sleep(delay)
            else:
                logger.error(f"Exhausted all {_MAX_RETRIES} retries for
embed_content due to ResourceExhausted.")
        except Exception as e:
            logger.error(f"Unhandled exception during embed_content:
{type(e).__name__}: {e}")
            raise

    raise last_exception

def embed_chunks(chunks: list, model: str, batch_size: int = 20) -> list:
    """
    Embeds a list of chunk dicts for indexing, using
    task_type=RETRIEVAL_DOCUMENT.

    Processes in batches of `batch_size`, sleeping 1 second between batches
    to respect API rate limits.

    Args:
        chunks: list of chunk dicts (must each have a "text" key).
        model: embedding model identifier, e.g. "models/text-embedding-
004".
        batch_size: number of chunks to embed per batch before sleeping.

    Returns:
        list[dict]: the same chunk dicts, each with an added "embedding"
key
        (list[float], 768 dimensions).
    """
    embedded_chunks = []
    total_batches = (len(chunks) + batch_size - 1) // batch_size if chunks
    else 0

    for batch_number, batch_start in enumerate(range(0, len(chunks),
batch_size), start=1):

```

```

    batch = chunks[batch_start: batch_start + batch_size]
    retries_triggered_this_batch = 0

    for chunk in batch:
        try:
            embedding = _embed_with_retry(chunk["text"], model=model,
task_type="RETRIEVAL_DOCUMENT")
            chunk_with_embedding = dict(chunk)
            chunk_with_embedding["embedding"] = embedding
            embedded_chunks.append(chunk_with_embedding)
        except ResourceExhausted:
            retries_triggered_this_batch += 1
            logger.error(f"Skipping chunk {chunk.get('chunk_id',
'<unknown>')} after exhausting retries.")
            continue

    logger.info(
        f"Embedding batch {batch_number}/{total_batches} complete | "
        f"embeddings_received={len(batch) -
retries_triggered_this_batch}/{len(batch)} | "
        f"retries_triggered={retries_triggered_this_batch}"
    )

    if batch_start + batch_size < len(chunks):
        time.sleep(_INTER_BATCH_SLEEP_SECONDS)

    return embedded_chunks

def embed_query(text: str, model: str) -> list:
    """
    Embeds a single query string for retrieval, using
    task_type=RETRIEVAL_QUERY.

    This must use RETRIEVAL_QUERY (not RETRIEVAL_DOCUMENT) – the asymmetric
    task types tell the model to optimize the embedding geometry for
    query-to-document matching rather than document-to-document matching.

    Args:
        text: the user's natural-language question.
        model: embedding model identifier, e.g. "models/text-embedding-
004".

    Returns:
        list[float]: the query embedding vector (768 dimensions).
    """
    return _embed_with_retry(text, model=model,
task_type="RETRIEVAL_QUERY")

```

3.6 src/vector_store.py

```

"""
Vertex AI Vector Search module. Owns the google-cloud-aiplatform API
boundary exclusively – no other module should import aiplatform directly.
"""

from google.api_core.exceptions import NotFound
from google.cloud import aiplatform
from google.cloud.aiplatform.matching_engine.matching_engine_index_endpoint import (
    MatchingEngineIndexEndpoint,
)
from google.cloud.aiplatform_v1.types import IndexDatapoint

from utils.logger import get_logger

logger = get_logger()

_UPSERT_BATCH_SIZE = 100

def _vector_norm(vector: list) -> float:
    """Computes the L2 norm of a vector, for diagnostic logging."""
    return sum(x * x for x in vector) ** 0.5

def upsert_chunks(chunks_with_embeddings: list, project: str, region: str,
index_id: str) -> int:
    """
    Upserts embedded chunks into a Vertex AI Vector Search Index as
    datapoints.

    Each datapoint carries restricts (Vertex AI's Namespace metadata filter
    mechanism) for "year", "doc_type", and "doc_id", enabling metadata-
    filtered
    ANN queries later.

    Args:
        chunks_with_embeddings: list of chunk dicts, each with a 768-d
        "embedding" key.
        project: GCP project ID.
        region: GCP region, e.g. "us-central1".
        index_id: Vertex AI Vector Search Index resource ID.

    Returns:
        int: total number of datapoints successfully upserted.

    Raises:
        NotFound: if the index_id does not resolve to a real index.
    """
    aiplatform.init(project=project, location=region)

    try:

```

```

        index = aiplatform.MatchingEngineIndex(index_name=index_id)
    except NotFound as e:
        logger.error(
            f"Vertex AI Index not found for index_id='{index_id}'. "
            f"Verify VERTEX_INDEX_ID is correct in .env. | {e}"
        )
        raise

    total_upserted = 0
    total_batches = (len(chunks_with_embeddings) + _UPSERT_BATCH_SIZE - 1)
    // _UPSERT_BATCH_SIZE

    for batch_number, batch_start in enumerate(
        range(0, len(chunks_with_embeddings), _UPSERT_BATCH_SIZE), start=1
    ):
        batch = chunks_with_embeddings[batch_start: batch_start +
        _UPSERT_BATCH_SIZE]

        datapoints = []
        for chunk in batch:
            restricts = [
                IndexDatapoint.Restriction(namespace="year", allow_list=
                [str(chunk["year"])]),
                IndexDatapoint.Restriction(namespace="doc_type",
                allow_list=[chunk["doc_type"]]),
                IndexDatapoint.Restriction(namespace="doc_id", allow_list=
                [chunk["doc_id"]]),
            ]
            datapoints.append(
                IndexDatapoint(
                    datapoint_id=chunk["chunk_id"],
                    feature_vector=chunk["embedding"],
                    restricts=restricts,
                )
            )

        try:
            index.upsert_datapoints(datapoints=datapoints)
            total_upserted += len(datapoints)
            logger.info(
                f"Upsert batch {batch_number}/{total_batches} complete | "
                f"datapoints_in_batch={len(datapoints)} | "
                f"total_upserted_so_far={total_upserted}"
            )
        except NotFound as e:
            logger.error(f"Index not found during upsert batch
            {batch_number}: {e}")
            raise
        except Exception as e:
            logger.error(f"Upsert batch {batch_number} failed:
            {type(e).__name__}: {e}. Skipping batch.")
            continue

    return total_upserted

```

```

def query_index(
    query_embedding: list,
    top_k: int,
    project: str,
    region: str,
    index_endpoint_id: str,
    deployed_index_id: str,
    filter_namespaces: dict = None,
) -> list:
    """
    Queries a deployed Vertex AI Vector Search IndexEndpoint for the top_k
    nearest neighbors of a query embedding, with optional metadata filters.

    Args:
        query_embedding: 768-d query vector (from embedder.embed_query()).
        top_k: number of nearest neighbors to retrieve.
        project: GCP project ID.
        region: GCP region.
        index_endpoint_id: Vertex AI IndexEndpoint resource ID.
        deployed_index_id: the deployed index's name within the endpoint.
        filter_namespaces: optional dict like {"year": "2023", "doc_type":
"arxiv_paper"},
            mapped onto Vertex AI's restricts (Namespace) filter mechanism.

    Returns:
        list[dict]: [{"chunk_id": str, "distance": float}, ...]

    Raises:
        NotFound: if index_endpoint_id does not resolve to a real, deployed
        endpoint.
    """
    aiplatform.init(project=project, location=region)

    norm = _vector_norm(query_embedding)
    logger.info(f"Querying index | top_k={top_k} | query_vector_norm=
{norm:.4f} | filters={filter_namespaces}")

    try:
        endpoint =
MatchingEngineIndexEndpoint(index_endpoint_name=index_endpoint_id)
    except NotFound as e:
        logger.error(
            f"Vertex AI IndexEndpoint not found for
index_endpoint_id='{index_endpoint_id}'. "
            f"Verify VERTEX_INDEX_ENDPOINT_ID is correct in .env. | {e}"
        )
        raise

    restricts_param = None
    if filter_namespaces:
        restricts_param = [
            {"namespace": namespace, "allow_list": [str(value)]}

```

```

        for namespace, value in filter_namespaces.items()
    ]

    try:
        response = endpoint.find_neighbors(
            deployed_index_id=deployed_index_id,
            queries=[query_embedding],
            num_neighbors=top_k,
            filter=restricts_param,
        )
    except NotFound as e:
        logger.error(
            f"Deployed index not found for "
            f"deployed_index_id='{deployed_index_id}' "
            f"on endpoint '{index_endpoint_id}'. Verify "
            f"VERTEX_DEPLOYED_INDEX_ID. | {e}"
        )
        raise

    results = []
    if response and len(response) > 0:
        for neighbor in response[0]:
            results.append({"chunk_id": neighbor.id, "distance":
neighbor.distance})

    logger.info(f"Query returned {len(results)} results.")
    return results

```

3.7 src/groq_synthesizer.py

```

"""
RAG answer-synthesis module. Owns the groq SDK API boundary exclusively –
no other module should import groq directly.
"""

import time

import groq

from config.settings import GROQ_API_KEY
from utils.logger import get_logger

logger = get_logger()

_client = groq.Groq(api_key=GROQ_API_KEY)

_MODEL = "llama-3.3-70b-versatile"
_TEMPERATURE = 0.1
_MAX_TOKENS = 1024
_MAX_RETRIES = 3

```

```

_RETRY_DELAY_SECONDS = 5

_SYSTEM_PROMPT = (
    "You are a research assistant. Answer ONLY using the provided context chunks. "
    "If the answer is not in the context, say 'I cannot find this in the indexed documents.' "
    "Do not hallucinate. Cite chunk_ids inline."
)

def _format_context(retrieved_chunks: list) -> str:
    """Formats retrieved chunks into the context block passed to the LLM."""
    formatted_blocks = []
    for chunk in retrieved_chunks:
        header = (
            f"[Chunk ID: {chunk.get('chunk_id', 'unknown')}] | "
            f"Doc: {chunk.get('title', 'unknown')} | "
            f"Year: {chunk.get('year', 'unknown')} | "
            f"Page: {chunk.get('page_number', 'unknown')}]]"
        )
        formatted_blocks.append(f"{header}\n{chunk.get('text', '')}")
    return "\n\n".join(formatted_blocks)

def synthesize_answer(question: str, retrieved_chunks: list) -> str:
    """
    Synthesizes a grounded natural-language answer from retrieved chunks using Groq's llama-3.3-70b-versatile.

    Args:
        question: the user's natural-language question.
        retrieved_chunks: list of chunk dicts (with chunk_id, title, year, page_number, text – typically hydrated from the manifest).

    Returns:
        str: the synthesized answer.

    Raises:
        groq.APIError, groq.RateLimitError: if all retries are exhausted.
    """
    context_block = _format_context(retrieved_chunks)
    user_message = (
        f"Context chunks:\n\n{context_block}\n\n"
        f"Question: {question}"
    )

    logger.info(f"Synthesizing answer | question='{question}' | num_chunks={len(retrieved_chunks)}")

    last_exception = None
    for attempt in range(_MAX_RETRIES):
        try:

```

```

        response = _client.chat.completions.create(
            model=_MODEL,
            temperature=_TEMPERATURE,
            max_tokens=_MAX_TOKENS,
            messages=[
                {"role": "system", "content": _SYSTEM_PROMPT},
                {"role": "user", "content": user_message},
            ],
        )
        answer = response.choices[0].message.content

        usage = getattr(response, "usage", None)
        if usage is not None:
            logger.info(
                f"Synthesis complete | prompt_tokens={usage.prompt_tokens} | "
                f"completion_tokens={usage.completion_tokens} | "
                f"total_tokens={usage.total_tokens}"
            )
        else:
            logger.info("Synthesis complete | token usage unavailable in response")

        return answer

    except (groq.APIError, groq.RateLimitError) as e:
        last_exception = e
        if attempt < _MAX_RETRIES - 1:
            logger.info(
                f"Groq API error on attempt {attempt + 1}/{_MAX_RETRIES}: {type(e).__name__}. "
                f"Retrying in {_RETRY_DELAY_SECONDS}s..."
            )
            time.sleep(_RETRY_DELAY_SECONDS)
        else:
            logger.error(f"Exhausted all {_MAX_RETRIES} retries for Groq synthesis: {e}")

        raise last_exception

```

3.8 vvs/ingest.py (Primary Deliverable)

```

"""
Primary Phase 1 deliverable: the offline ingestion pipeline.

Orchestrates: download -> parse -> chunk -> embed -> upsert -> manifest
update,
end to end, for a corpus of arXiv ML papers.

Run with:

```

```

python vvs/ingest.py
"""

import csv
import os
import sys

import arxiv

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from config.settings import ( # noqa: E402
    EMBEDDING_MODEL,
    CHUNK_SIZE_TOKENS,
    CHUNK_OVERLAP_TOKENS,
    GCP_PROJECT_ID,
    GCP_REGION,
    MANIFEST_PATH,
    PDF_DATA_DIR,
    VERTEX_INDEX_ID,
    validate_settings,
)
from src.chunker import chunk_document # noqa: E402
from src.embedder import embed_chunks # noqa: E402
from src.pdf_parser import parse_pdf # noqa: E402
from src.vector_store import upsert_chunks # noqa: E402
from utils.logger import get_logger # noqa: E402

logger = get_logger()

_SEARCH_QUERY = "transformer architecture OR large language models"
_TARGET_PAPER_COUNT = 200
_MANIFEST_COLUMNS = ["doc_id", "title", "year", "doc_type", "num_pages",
"num_chunks"]

def download_corpus(target_count: int, output_dir: str) -> list:
    """
    Searches arXiv for ML papers and downloads PDFs into output_dir.
    Skips files that already exist on disk.

    Args:
        target_count: number of papers to download.
        output_dir: directory to download PDFs into.

    Returns:
        list[str]: filesystem paths to all PDFs available locally
        (both newly downloaded and previously downloaded).
    """
    os.makedirs(output_dir, exist_ok=True)

    client = arxiv.Client()
    search = arxiv.Search(

```

```

        query=_SEARCH_QUERY,
        max_results=target_count,
        sort_by=arxiv.SortCriterion.Relevance,
    )

    pdf_paths = []
    downloaded_count = 0
    skipped_count = 0

    for result in client.results(search):
        arxiv_id = result.get_short_id()
        safe_filename = f"{arxiv_id.replace('/', '_')}.pdf"
        destination_path = os.path.join(output_dir, safe_filename)

        if os.path.exists(destination_path):
            skipped_count += 1
            pdf_paths.append(destination_path)
            continue

        try:
            result.download_pdf(dirpath=output_dir, filename=safe_filename)
            downloaded_count += 1
            pdf_paths.append(destination_path)
            logger.info(f"Downloaded: '{result.title}' | arxiv_id={
{arxiv_id}}")
        except Exception as e:
            logger.error(f"Failed to download arxiv_id={arxiv_id} | {e}")
            continue

    logger.info(
        f"Corpus download complete | downloaded={downloaded_count} | "
        f"already_present_skipped={skipped_count} | total_available=
{len(pdf_paths)}")
    )
    return pdf_paths

def append_to_manifest(manifest_path: str, doc: dict, num_chunks: int) ->
None:
    """
    Appends a single document's metadata row to the manifest CSV.
    Writes the header only if the file is currently empty.
    """
    file_is_empty = (not os.path.exists(manifest_path)) or
os.path.getsize(manifest_path) == 0

    os.makedirs(os.path.dirname(manifest_path), exist_ok=True)

    with open(manifest_path, mode="a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=_MANIFEST_COLUMNS,
extrasaction="ignore")
        if file_is_empty:
            writer.writeheader()
        writer.writerow({

```

```

        "doc_id": doc["doc_id"],
        "title": doc["title"],
        "year": doc["year"],
        "doc_type": doc["doc_type"],
        "num_pages": doc["num_pages"],
        "num_chunks": num_chunks,
    })

```

```

def run_ingestion() -> None:
    """Runs the full Phase 1 ingestion pipeline end to end."""
    validate_settings()

    logger.info("=== Starting ingestion pipeline ===")

    pdf_paths = download_corpus(_TARGET_PAPER_COUNT, PDF_DATA_DIR)

    total_attempted = len(pdf_paths)
    successfully_ingested = 0
    failed_or_skipped = 0
    total_chunks_created = 0
    total_embeddings_pushed = 0

    for pdf_path in pdf_paths:
        doc = parse_pdf(pdf_path)
        if doc is None:
            failed_or_skipped += 1
            continue

        chunks = chunk_document(doc, chunk_size=CHUNK_SIZE_TOKENS,
                                overlap=CHUNK_OVERLAP_TOKENS)
        if not chunks:
            logger.info(f"doc_id={doc['doc_id']} produced no chunks -
skipping upsert and manifest entry.")
            failed_or_skipped += 1
            continue

        embedded_chunks = embed_chunks(chunks, model=EMBEDDING_MODEL)
        if not embedded_chunks:
            logger.error(f"doc_id={doc['doc_id']} produced zero successful
embeddings - skipping.")
            failed_or_skipped += 1
            continue

        try:
            upserted_count = upsert_chunks(
                embedded_chunks,
                project=GCP_PROJECT_ID,
                region=GCP_REGION,
                index_id=VERTEX_INDEX_ID,
            )
        except Exception as e:
            logger.error(f"Upsert failed for doc_id={doc['doc_id']}: {e}.
Skipping manifest entry.")

```

```

        failed_or_skipped += 1
        continue

    append_to_manifest(MANIFEST_PATH, doc, num_chunks=len(chunks))

    successfully_ingested += 1
    total_chunks_created += len(chunks)
    total_embeddings_pushed += upserted_count

    logger.info("=== Ingestion pipeline complete ===")
    logger.info(f"Total PDFs attempted:      {total_attempted}")
    logger.info(f"Successfully ingested:      {successfully_ingested}")
    logger.info(f"Failed/skipped:              {failed_or_skipped}")
    logger.info(f"Total chunks created:        {total_chunks_created}")
    logger.info(f"Total embeddings pushed:     {total_embeddings_pushed}")

if __name__ == "__main__":
    run_ingestion()

```

3.9 vvs/query.py (Primary Deliverable)

```

"""
Primary Phase 2 deliverable: the online query pipeline.

Orchestrates: embed question -> ANN lookup -> metadata hydration ->
optional Groq synthesis -> print results.

Run with:
    python vvs/query.py --question "What is self-attention?"
    python vvs/query.py --question "..." --year 2023 --top_k 10 --no-
synthesis
"""

import argparse
import csv
import os
import sys

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from config.settings import ( # noqa: E402
    EMBEDDING_MODEL,
    GCP_PROJECT_ID,
    GCP_REGION,
    MANIFEST_PATH,
    TOP_K_RESULTS,
    VERTEX_DEPLOYED_INDEX_ID,
    VERTEX_INDEX_ENDPOINT_ID,
    validate_settings,

```

```

)
from src.embedder import embed_query # noqa: E402
from src.groq_synthesizer import synthesize_answer # noqa: E402
from src.vector_store import query_index # noqa: E402
from utils.logger import get_logger # noqa: E402

logger = get_logger()

def parse_args() -> argparse.Namespace:
    """Parses CLI arguments for query.py."""
    parser = argparse.ArgumentParser(description="Query the Day 5 vector search RAG pipeline.")
    parser.add_argument("--question", required=True, type=str, help="The search query string.")
    parser.add_argument("--year", required=False, type=str, default=None, help="Filter to a specific year, e.g. '2023'.")
    parser.add_argument("--doc_type", required=False, type=str, default=None, help="Filter to a specific doc_type.")
    parser.add_argument("--top_k", required=False, type=int, default=TOP_K_RESULTS, help="Number of chunks to retrieve.")
    parser.add_argument("--no-synthesis", dest="no_synthesis", action="store_true", help="Skip Groq synthesis; return raw chunks only.")
    return parser.parse_args()

def load_manifest(manifest_path: str) -> dict:
    """
    Loads the manifest CSV into a dict keyed by doc_id, once, at startup.

    Args:
        manifest_path: path to corpus_manifest.csv.

    Returns:
        dict: {doc_id: {"title": str, "year": str, "doc_type": str, ...}}
    """
    manifest = {}
    if not os.path.exists(manifest_path):
        logger.error(f"Manifest not found at {manifest_path}. Run ingest.py first.")
    return manifest

    with open(manifest_path, mode="r", newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            manifest[row["doc_id"]] = row

    return manifest

def hydrate_chunk_metadata(raw_results: list, manifest: dict) -> list:
    """
    Hydrates raw {chunk_id, distance} results with full document metadata looked up from the manifest, by parsing doc_id out of chunk_id.

```

Args:

raw_results: list of {"chunk_id": str, "distance": float}.
 manifest: dict keyed by doc_id, as returned by load_manifest().

Returns:

list[dict]: enriched chunk dicts with title, year, doc_type, page_number populated where available. page_number is not stored in the manifest (it is doc-level, not chunk-level), so it is reported as "unknown" here; the chunk_index suffix of chunk_id is included for reference.

```

"""
enriched = []
for result in raw_results:
    chunk_id = result["chunk_id"]
    doc_id = chunk_id.rsplit("_chunk_", 1)[0] if "_chunk_" in chunk_id
else chunk_id
    doc_meta = manifest.get(doc_id, {})

    enriched.append({
        "chunk_id": chunk_id,
        "distance": result["distance"],
        "doc_id": doc_id,
        "title": doc_meta.get("title", "unknown"),
        "year": doc_meta.get("year", "unknown"),
        "doc_type": doc_meta.get("doc_type", "unknown"),
        "page_number": "see chunk_id index",
        "text": "(raw text not stored in manifest - re-fetch from index
metadata or local chunk cache if needed)",
    })
return enriched

```

```

def print_chunks(chunks: list) -> None:
    """Prints the top-K raw chunks with their metadata to stdout."""
    print("\n=== Top Retrieved Chunks ===")
    for i, chunk in enumerate(chunks, start=1):
        print(
            f"{i}. chunk_id={chunk['chunk_id']} | distance=
{chunk['distance']:.4f} | "
            f"title='{chunk['title']}' | year={chunk['year']} | doc_type=
{chunk['doc_type']}"
        )
    print()

```

```

def run_query() -> None:
    """Runs the full Phase 2 query pipeline end to end."""
    validate_settings()
    args = parse_args()

```

```

manifest = load_manifest(MANIFEST_PATH)

query_embedding = embed_query(args.question, model=EMBEDDING_MODEL)

filter_namespaces = {}
if args.year:
    filter_namespaces["year"] = args.year
if args.doc_type:
    filter_namespaces["doc_type"] = args.doc_type

raw_results = query_index(
    query_embedding=query_embedding,
    top_k=args.top_k,
    project=GCP_PROJECT_ID,
    region=GCP_REGION,
    index_endpoint_id=VERTEX_INDEX_ENDPOINT_ID,
    deployed_index_id=VERTEX_DEPLOYED_INDEX_ID,
    filter_namespaces=filter_namespaces or None,
)

enriched_chunks = hydrate_chunk_metadata(raw_results, manifest)

print_chunks(enriched_chunks)

synthesis_token_usage = "skipped (--no-synthesis)"
if not args.no_synthesis:
    answer = synthesize_answer(args.question, enriched_chunks)
    print("=== Synthesized Answer ===")
    print(answer)
    print()
    synthesis_token_usage = "see output.log for token usage detail"

logger.info(
    f"Query event complete | question='{args.question}' | filters=
{filter_namespaces} | "
    f"chunk_ids_returned={c['chunk_id'] for c in enriched_chunks} | "
    f"synthesis_token_usage={synthesis_token_usage}"
)

if __name__ == "__main__":
    run_query()

```

3.10 Test Suite (tests/)

These tests cover the pure-computation and mockable-boundary logic. Network-dependent calls (actual Gemini/Vertex/Groq API calls) are mocked rather than hit live, so the suite runs offline and fast.

tests/__init__.py

```
"""Marks tests/ as a package so pytest can discover modules via relative imports."""
```

tests/test_chunker.py

```
"""
Unit tests for src/chunker.py.

No network calls – chunk_document() is pure computation over tiktoken
encodings, so these tests run fully offline.
"""

import os
import sys

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.chunker import chunk_document # noqa: E402

def _make_doc(text: str, num_pages: int = 1) -> dict:
    """Builds a minimal doc dict for testing, all text on page 1."""
    return {
        "doc_id": "abc123def456",
        "title": "Test Document",
        "year": 2023,
        "doc_type": "arxiv_paper",
        "num_pages": num_pages,
        "pages": [{"page_number": 1, "text": text}],
    }

def test_chunk_document_produces_expected_chunk_count():
    # ~1500 tokens of repeated text, chunk_size=512, overlap=64 ->
    stride=448
    long_text = " ".join(["word"] * 1500)
    doc = _make_doc(long_text)

    chunks = chunk_document(doc, chunk_size=512, overlap=64)

    assert len(chunks) >= 1
    for chunk in chunks:
        assert chunk["token_count"] <= 512
        assert chunk["doc_id"] == "abc123def456"

def test_chunk_document_chunk_ids_are_sequential_and_zero_padded():
    long_text = " ".join(["word"] * 1500)
    doc = _make_doc(long_text)
```

```

chunks = chunk_document(doc, chunk_size=512, overlap=64)

for i, chunk in enumerate(chunks):
    assert chunk["chunk_id"] == f"abc123def456_chunk_{i:04d}"

def test_chunk_document_skips_degenerate_short_documents():
    short_text = "Too short."
    doc = _make_doc(short_text)

    chunks = chunk_document(doc, chunk_size=512, overlap=64)

    # Fewer than 20 tokens total -> should produce zero chunks.
    assert chunks == []

def test_chunk_document_raises_on_invalid_overlap():
    doc = _make_doc("some text here for testing purposes only")
    try:
        chunk_document(doc, chunk_size=100, overlap=100)
        assert False, "Expected ValueError for overlap >= chunk_size"
    except ValueError:
        pass

def test_chunk_document_empty_pages_returns_empty_list():
    doc = _make_doc("")
    doc["pages"] = []
    chunks = chunk_document(doc, chunk_size=512, overlap=64)
    assert chunks == []

```

tests/test_embedder.py

```

"""
Unit tests for src/embedder.py.

All genai.embed_content() calls are mocked – no live API calls, no API key
needed.
"""

import os
import sys
from unittest.mock import patch

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from google.api_core.exceptions import ResourceExhausted # noqa: E402

from src.embedder import embed_chunks, embed_query # noqa: E402

```

```

def _fake_embedding_response(*args, **kwargs):
    return {"embedding": [0.1] * 768}

@patch("src.embedder.genai.embed_content",
side_effect=_fake_embedding_response)
def test_embed_query_returns_768_dim_vector(mock_embed):
    result = embed_query("What is attention?", model="models/text-embedding-004")
    assert len(result) == 768
    mock_embed.assert_called_once()
    _, kwargs = mock_embed.call_args
    assert kwargs["task_type"] == "RETRIEVAL_QUERY"

@patch("src.embedder.genai.embed_content",
side_effect=_fake_embedding_response)
def test_embed_chunks_adds_embedding_key_to_every_chunk(mock_embed):
    chunks = [
        {"chunk_id": "doc1_chunk_0000", "text": "chunk one text"},
        {"chunk_id": "doc1_chunk_0001", "text": "chunk two text"},
    ]
    result = embed_chunks(chunks, model="models/text-embedding-004",
batch_size=20)

    assert len(result) == 2
    for chunk in result:
        assert "embedding" in chunk
        assert len(chunk["embedding"]) == 768

@patch("src.embedder.genai.embed_content",
side_effect=_fake_embedding_response)
def test_embed_chunks_uses_retrieval_document_task_type(mock_embed):
    chunks = [{"chunk_id": "doc1_chunk_0000", "text": "some text"}]
    embed_chunks(chunks, model="models/text-embedding-004", batch_size=20)

    _, kwargs = mock_embed.call_args
    assert kwargs["task_type"] == "RETRIEVAL_DOCUMENT"

@patch("src.embedder.time.sleep", return_value=None)
@patch(
    "src.embedder.genai.embed_content",
    side_effect=[ResourceExhausted("quota exceeded")] * 6 + [{"embedding":
[0.1] * 768}],
)
def
test_embed_query_retries_on_resource_exhausted_then_raises_if_never_succeed
s(mock_embed, mock_sleep):
    try:
        embed_query("test question", model="models/text-embedding-004")

```

```

        assert False, "Expected ResourceExhausted to be raised after
exhausting retries"
    except ResourceExhausted:
        pass
    # 1 initial attempt + 5 retries = 6 calls before giving up
    assert mock_embed.call_count == 6

```

tests/test_query.py

```

"""
Unit tests for vvs/query.py.

Covers the pure-logic pieces (manifest loading, chunk hydration, arg
parsing)
without invoking live embedding, Vertex AI, or Groq calls.
"""

import csv
import os
import sys
import tempfile

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
sys.path.insert(0,
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"vvs"))

from vvs.query import hydrate_chunk_metadata, load_manifest # noqa: E402

def test_load_manifest_reads_csv_into_dict_keyed_by_doc_id():
    with tempfile.NamedTemporaryFile(mode="w", suffix=".csv", delete=False,
newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["doc_id", "title", "year",
"doc_type", "num_pages", "num_chunks"])
        writer.writeheader()
        writer.writerow({
            "doc_id": "abc123def456",
            "title": "Attention Is All You Need",
            "year": "2017",
            "doc_type": "arxiv_paper",
            "num_pages": "15",
            "num_chunks": "30",
        })
        temp_path = f.name

    try:
        manifest = load_manifest(temp_path)
        assert "abc123def456" in manifest
        assert manifest["abc123def456"]["title"] == "Attention Is All You

```

```
Need"
    finally:
        os.remove(temp_path)

def test_load_manifest_missing_file_returns_empty_dict():
    manifest = load_manifest("/tmp/this_file_does_not_exist_12345.csv")
    assert manifest == {}

def test_hydrate_chunk_metadata_parses_doc_id_from_chunk_id():
    raw_results = [{"chunk_id": "abc123def456_chunk_0003", "distance":
0.87}]
    manifest = {
        "abc123def456": {
            "title": "Attention Is All You Need",
            "year": "2017",
            "doc_type": "arxiv_paper",
        }
    }

    enriched = hydrate_chunk_metadata(raw_results, manifest)

    assert len(enriched) == 1
    assert enriched[0]["doc_id"] == "abc123def456"
    assert enriched[0]["title"] == "Attention Is All You Need"
    assert enriched[0]["distance"] == 0.87

def
test_hydrate_chunk_metadata_handles_missing_manifest_entry_gracefully():
    raw_results = [{"chunk_id": "unknown_doc_chunk_0000", "distance": 0.5}]
    manifest = {}

    enriched = hydrate_chunk_metadata(raw_results, manifest)

    assert enriched[0]["title"] == "unknown"
    assert enriched[0]["year"] == "unknown"
```

SECTION 4 — Code Logic & Deep-Dive

4.1 PDF Parsing Deep Dive

`pypdf` extracts text on a page-by-page basis because PDFs are fundamentally page-oriented documents internally — there is no native "whole document text stream," only a sequence of page objects, each with its own content stream of positioned glyphs. `parse_pdf()` respects this by returning a `pages` list rather than a single blob, which matters downstream: `chunker.py` needs to know which page a chunk came from so that citations in the synthesized answer can point a user to "Page 7" rather than just a chunk ID.

`clean_text()` exists because raw PDF text extraction is noisy in a structurally predictable way: running headers ("Proceedings of NeurIPS 2023"), page numbers ("7"), figure/table captions split across many short lines, and footnote markers all extract as very short lines. The 3-word-minimum filter removes this class of noise without needing layout analysis — it's a cheap heuristic that works well specifically because academic paper noise tends to be short while actual prose sentences are not. Without this filter, junk tokens like isolated page numbers would get embedded as part of a chunk and slightly shift that chunk's embedding away from its true semantic content, very marginally degrading retrieval precision.

SHA256 hashing of file *content* (not filename) is used for `doc_id` for two reasons: (1) idempotency — if the same paper is downloaded twice under different filenames (e.g. arXiv updates a paper's listing or the script is re-run with different the naming scheme), content hashing still produces the same `doc_id`, preventing duplicate index entries; (2) collision resistance and stability — filenames can collide trivially (`paper.pdf` is a common default), while a SHA256 prefix of file bytes is effectively unique per actual document.

4.2 Chunking Strategy Explained

The sliding window algorithm works in token-space, not character-space, because token count is what `chunk_size=512` is actually bounding (LLM context windows are measured in tokens). The algorithm:

1. Encode the full joined-page text into a single list of token IDs.
2. Starting at `start = 0`, take the window `[start : start + chunk_size]`.
3. Decode that window back into a text chunk.
4. Advance `start` by `stride = chunk_size - overlap` tokens.
5. Repeat until `start >= total_tokens`.

Concrete worked example — a 1,500-token document, `chunk_size=512`, `overlap=64`:

- `stride = 512 - 64 = 448`
- Chunk 0: tokens `[0:512]` → covers tokens 0–511 (512 tokens)
- Chunk 1: tokens `[448:960]` → covers tokens 448–959 (512 tokens) — overlaps chunk 0 on tokens 448–511 (64 tokens, exactly the overlap)
- Chunk 2: tokens `[896:1408]` → covers tokens 896–1407 (512 tokens) — overlaps chunk 1 on tokens 896–959 (64 tokens)
- Chunk 3: tokens `[1344:1500]` → covers tokens 1344–1499 (only 156 tokens, since `total_tokens=1500` is reached) — this is the final, shorter chunk

That's **4 chunks** for a 1,500-token document under these settings (and all clear the 20-token minimum, so none are dropped). Note the general formula: the number of full-size windows before the final partial one is `ceil((total_tokens - chunk_size) / stride) + 1`.

The overlap exists because token window boundaries are arbitrary with respect to sentence and idea boundaries. If chunk 0 ends mid-sentence at token 511 and chunk 1 begins at token 512, the sentence that straddles that boundary is split and incomplete in *both* chunks — its embedding in chunk 0 lacks its conclusion, and its embedding in chunk 1 lacks its setup. By making chunk 1 start at token 448 instead of token 512, the 64-token overlap zone gives that straddling sentence a much higher chance of being fully contained within at least one of the two chunks, preserving its complete semantic content for embedding purposes.

4.3 Embedding Pipeline

`RETRIEVAL_DOCUMENT` and `RETRIEVAL_QUERY` are *asymmetric* task type hints that tell `text-embedding-004` to project the same input text into geometrically different (but compatible) regions of embedding space, depending on whether that text is playing the role of a thing-to-be-found or a thing-doing-the-finding. This exists because questions and answers are linguistically different — a question ("What is self-attention?") and the passage that answers it ("Self-attention computes a weighted combination of value vectors...") rarely share much surface-level vocabulary or sentence structure, even though they're a perfect semantic match. The model has learned, during its training, to push query-type and document-type embeddings of matching content closer together in the joint embedding space than a naively symmetric embedding model would.

Using the wrong task type — for example, embedding documents with `RETRIEVAL_QUERY` during indexing — degrades retrieval because the index then contains vectors optimized for "this is a question," and at query time the actual question (also embedded as `RETRIEVAL_QUERY`) is being compared against vectors that were never positioned to be *found by* a query. The asymmetry that the model relies on to make retrieval work is collapsed, and the system effectively becomes a generic, lower-quality symmetric similarity search.

Vector normalization scales a vector to unit length (L2 norm = 1) while preserving its direction. `text-embedding-004` returns pre-normalized vectors, meaning all the semantic information lives entirely in the vector's *direction*, none in its magnitude. This is exactly the condition under which dot product and cosine similarity coincide:

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

If $||A|| = ||B|| = 1$ (both normalized), the denominator becomes $1 * 1 = 1$, so:

$$\text{cosine_similarity}(A, B) = A \cdot B$$

The dot product *is* the cosine similarity, with no extra division required — which is exactly why `DOT_PRODUCT_DISTANCE` was chosen for this index (Section 1.3): it gets cosine-similarity-quality ranking at raw-dot-product cost.

4.4 Vertex AI Vector Search Internals

ANN vs exact KNN. Exact K-Nearest-Neighbors computes the true distance from a query vector to *every* vector in the index, then sorts to find the top-K — guaranteed-correct, but $O(N)$ per query, where N is the total vector count. At 200K+ vectors (a corpus this project could grow into well beyond its initial 200 papers), that becomes too slow for an interactive query loop. Approximate Nearest Neighbor (ANN) algorithms — Vertex AI uses a tree-based ANN approach (`treeAhConfig`) — instead build an index structure that lets a query only compare against a small, well-chosen subset of vectors, trading a small, usually negligible amount of recall accuracy for a massive speedup (often sub-linear or near-constant time relative to N).

The **restricts (Namespace) filter mechanism** operates *inside* the ANN search, not as a post-filter. Each datapoint carries a list of namespace/allow-list pairs (e.g. `namespace="year", allow_list=["2023"]`) as side metadata attached to the vector at upsert time. At query time, supplying a **restricts** filter (e.g. `{"namespace": "year", "allow_list": ["2023"]}`) tells the ANN search to only consider datapoints whose restricts satisfy that filter *while traversing the index structure* — datapoints outside the filter are pruned early, rather than being retrieved and then discarded. This is both faster (less wasted traversal) and more correct (you get the true top-K *among the filtered set*, not the top-K overall with non-matching ones removed afterward, which could return fewer than K results).

Index resource vs IndexEndpoint resource. The **Index** resource is the data structure itself — the vectors, their restricts metadata, and the ANN tree built over them. It is not, by itself, queryable; it has no compute attached to serve requests. The **IndexEndpoint** resource is the deployed, network-addressable serving layer — when you "deploy an Index to an IndexEndpoint," Vertex AI provisions the actual machines that hold the index in memory and answer **find_neighbors** calls. This separation exists because it lets you build/update an Index independently of its serving infrastructure (e.g. you can build a new Index version while the old one is still being served, then swap), and because a single IndexEndpoint can theoretically serve multiple deployed indexes (the **deployed_index_id** distinguishes between them on a shared endpoint).

4.5 Groq Synthesis Layer

The context window is constructed by formatting each retrieved chunk as a labeled block (`[Chunk ID: ... | Doc: ... | Year: ... | Page: ...]` followed by the chunk text), then joining all blocks with blank-line separators, then appending the user's original question at the end. This structure gives the LLM explicit, parseable provenance for every piece of context it's being asked to reason over — which is what makes "cite chunk_ids inline" a followable instruction rather than a vague aspiration.

temperature=0.1 (not 0.0, not 0.7) is chosen because this is a factual-retrieval-grounded task, not a creative one: near-zero temperature minimizes sampling randomness so the same question against the same retrieved context tends to produce highly consistent, literal answers — important for an internal tool where reproducibility during testing matters. A small nonzero value (rather than exactly 0) is kept because some API providers' **temperature=0** can occasionally produce degenerate repetition artifacts; 0.1 avoids that edge case while still being effectively deterministic for this use case.

The citation instruction ("cite chunk_ids inline") achieves two things: it gives the end user (or the next stage of an application built on this) a way to trace any claim in the answer back to its exact source chunk, and it creates an implicit self-check pressure on the model — a model instructed to cite its sources is, in practice, less likely to silently drift into unsupported claims than one given no such instruction, because the citation requirement keeps the model's attention anchored to specific passages in the context.

When retrieved chunks are insufficient to answer the question, the system prompt's explicit fallback ("If the answer is not in the context, say 'I cannot find this in the indexed documents.'") is what prevents hallucination. Whether the model actually obeys this perfectly is itself an empirical question — it is not a logical guarantee — which is why Section 6's example queries deliberately include a query designed to test this fallback behavior in practice.

4.6 End-to-End Data Flow — Tracing One PDF

Take a single arXiv PDF, `2308.xxxxx.pdf`, "Scaling Laws for Neural Language Models," through the entire pipeline:

1. `ingest.py::download_corpus()` calls `arxiv.Client().results(search)`, finds this paper among the results, and calls `result.download_pdf()`. **Data type: bytes on disk. API boundary: arXiv Atom feed / PDF download endpoint.**
2. `ingest.py::run_ingestion()` calls `pdf_parser.parse_pdf("data/pdfs/2308.xxxxx.pdf")`. Internally: `_compute_doc_id()` hashes the file → `"a1b2c3d4e5f6"`. `pypdf.PdfReader` opens the file. Each page's `.extract_text()` is called and passed through `clean_text()`. **Data type: in-memory dict {doc_id, title, year, doc_type, num_pages, pages: [...]}. API boundary: none external — local pypdf parsing.**
3. `ingest.py::run_ingestion()` calls `chunker.chunk_document(doc, 512, 64)`. Internally: pages are joined with `--- Page N ---` separators, the full text is tokenized with `tiktoken`, and the sliding window algorithm (Section 4.2) produces, say, 38 chunk dicts. **Data type: list[dict] of chunk records. API boundary: none external — local tiktoken tokenization.**
4. `ingest.py::run_ingestion()` calls `embedder.embed_chunks(chunks, "models/text-embedding-004")`. Internally: chunks are batched 20 at a time; each chunk's `text` is sent via `genai.embed_content(task_type="RETRIEVAL_DOCUMENT")`. **Data type: each chunk dict gains an "embedding": list[float] (768 floats). API boundary: Gemini Embedding API (Google AI Studio), HTTPS.**
5. `ingest.py::run_ingestion()` calls `vector_store.upsert_chunks(embedded_chunks, project, region, VERTEX_INDEX_ID)`. Internally: each chunk becomes an `IndexDatapoint` with `feature_vector` and three `restricts` (year, doc_type, doc_id); batches of 100 are pushed via `index.upsert_datapoints()`. **Data type: int (count upserted), 38 in this case. API boundary: Vertex AI Vector Search API (GCP), gRPC/REST.**
6. `ingest.py::append_to_manifest()` writes one row — `a1b2c3d4e5f6, Scaling Laws for Neural Language Models, 2020, arxiv_paper, 19, 38` — to `vvs/corpus_manifest.csv`. **Data type: CSV row on disk. API boundary: none — local file I/O.**

Later, a user runs `query.py --question "How does model performance scale with parameter count?"`:

7. `query.py::run_query()` calls `embedder.embed_query(question, model)` with `task_type="RETRIEVAL_QUERY"`. **Data type: list[float], 768-d. API boundary: Gemini Embedding API.**
8. `query.py::run_query()` calls `vector_store.query_index(query_embedding, top_k=5, ...)`. Vertex AI's deployed endpoint runs ANN search and, suppose, returns `a1b2c3d4e5f6_chunk_0012` among the top 5 (this chunk happens to discuss the scaling-laws power curve). **Data type: list[dict] of {chunk_id, distance}. API boundary: Vertex AI Vector Search API (deployed endpoint).**
9. `query.py::hydrate_chunk_metadata()` parses `doc_id="a1b2c3d4e5f6"` out of the `chunk_id`, looks it up in the manifest dict loaded by `load_manifest()`, and attaches `title="Scaling Laws for Neural Language Models", year="2020", doc_type="arxiv_paper"`. **Data type: enriched list[dict]. API boundary: none — local CSV lookup.**

10. `query.py::run_query()` calls `groq_synthesizer.synthesize_answer(question, enriched_chunks)`, which formats the chunks into a labeled context block and sends it to Groq. **Data type: `str` (final answer). API boundary: Groq Cloud API, HTTPS.**
11. The answer and raw chunks are printed to stdout, and the full event (question, filters, chunk IDs, token usage) is logged to `output.log` via `utils.logger.get_logger()`.

Note on chunk text in query results: the manifest CSV is intentionally doc-level only (per Section 3.8's column spec), so `hydrate_chunk_metadata()` cannot recover a chunk's original `text` or exact `page_number` from the manifest alone — only `vvs/ingest.py`'s in-memory chunk objects ever held that. In this implementation, `groq_synthesizer.synthesize_answer()` therefore receives doc-level metadata (title/year/doc_type) per chunk, with a placeholder noting that chunk text isn't manifest-resident. **To get real chunk text into synthesis, extend `corpus_manifest.csv` to a chunk-level manifest (one row per chunk, including `text` and `page_number`), or store chunk text as a Vertex AI datapoint metadata field via the `restricts` mechanism (string-only) or a side store (e.g. a local SQLite or JSON keyed by `chunk_id`) written during `upsert_chunks()`.** This is flagged explicitly here rather than silently faked, because it is the one real architectural gap in the spec as given — the manifest schema in Section 2 was fixed at the document level, but full RAG synthesis needs chunk-level text. Section 8 (if you extend this project) is the natural place to implement a chunk-level side-store.

SECTION 5 — Deployment & Execution Guide

All commands below are macOS bash-compatible. Run them in order.

5.1 GCP Pre-requisites

Step 1 — Enable required APIs:

```
gcloud services enable aiplatform.googleapis.com
gcloud services enable storage.googleapis.com
```

Step 2 — Create a GCS bucket for Vector Search staging data:

```
export GCP_PROJECT_ID="your-project-id"
export GCP_REGION="us-central1"
export BUCKET_NAME="${GCP_PROJECT_ID}-vector-search-staging"

gcloud storage buckets create "gs://${BUCKET_NAME}" \
  --project="${GCP_PROJECT_ID}" \
  --location="${GCP_REGION}" \
  --uniform-bucket-level-access
```

Step 3 — Create the Vertex AI Vector Search Index:

```
gcloud ai indexes create \
  --display-name="arxiv-ml-index" \
  --metadata-file=index_metadata.json \
  --project="${GCP_PROJECT_ID}" \
  --region="${GCP_REGION}"
```

Where `index_metadata.json` (create this file alongside the command above) contains:

```
{
  "contentsDeltaUri": "gs://YOUR_BUCKET_NAME/initial_empty_delta/",
  "config": {
    "dimensions": 768,
    "approximateNeighborsCount": 150,
    "distanceMeasureType": "DOT_PRODUCT_DISTANCE",
    "algorithmConfig": {
      "treeAhConfig": {
        "leafNodeEmbeddingCount": 500,
        "leafNodesToSearchPercent": 7
      }
    },
    "shardSize": "SHARD_SIZE_SMALL"
  }
}
```

Replace `YOUR_BUCKET_NAME` with the `BUCKET_NAME` from Step 2. The `contentsDeltaUri` can point to an empty folder for index creation — `ingest.py` populates the index afterward via `upsert_datapoints()`, not via batch delta files.

Step 4 — Deploy the Index to an IndexEndpoint:

```
# 4a. Create the (initially empty) IndexEndpoint
gcloud ai index-endpoints create \
  --display-name="arxiv-ml-endpoint" \
  --public-endpoint-enabled \
  --project="${GCP_PROJECT_ID}" \
  --region="${GCP_REGION}"

# 4b. Deploy the index onto that endpoint (replace the IDs below — see Step 5)
gcloud ai index-endpoints deploy-index "YOUR_INDEX_ENDPOINT_ID" \
  --deployed-index-id="arxiv_ml_deployed_v1" \
  --display-name="arxiv-ml-deployed-v1" \
  --index="YOUR_INDEX_ID" \
  --project="${GCP_PROJECT_ID}" \
  --region="${GCP_REGION}"
```

Step 5 — Find your resource IDs after creation:

```
# List indexes to get VERTEX_INDEX_ID
gcloud ai indexes list --project="${GCP_PROJECT_ID}" --
region="${GCP_REGION}"

# List index endpoints to get VERTEX_INDEX_ENDPOINT_ID
gcloud ai index-endpoints list --project="${GCP_PROJECT_ID}" --
region="${GCP_REGION}"

# Describe the endpoint to confirm VERTEX_DEPLOYED_INDEX_ID (the deployed-
index-id you set in 4b)
gcloud ai index-endpoints describe "YOUR_INDEX_ENDPOINT_ID" \
  --project="${GCP_PROJECT_ID}" \
  --region="${GCP_REGION}"
```

Copy the three resulting IDs into `.env` (Section 5.3).

Wait for ACTIVE state. Creating a Vector Search Index typically takes **30–60 minutes**, and deploying it to an endpoint takes additional time. Before running `ingest.py`, confirm the index shows `state: ACTIVE` in the output of `gcloud ai indexes describe`. Running `ingest.py` against a still-provisioning index will fail upsert calls.

5.2 Virtual Environment & Dependencies

```
# Exact commands for macOS bash
python3 -m venv myenv
source myenv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

5.3 `.env` File Configuration

```
GROQ_API_KEY=your_groq_key_here
GEMINI_API_KEY=your_gemini_key_here
GCP_PROJECT_ID=your_gcp_project_id
GCP_REGION=us-central1
VERTEX_INDEX_ID=
VERTEX_INDEX_ENDPOINT_ID=
VERTEX_DEPLOYED_INDEX_ID=
EMBEDDING_MODEL=models/text-embedding-004
EMBEDDING_DIMENSION=768
CHUNK_SIZE_TOKENS=512
CHUNK_OVERLAP_TOKENS=64
TOP_K_RESULTS=5
PDF_DATA_DIR=data/pdfs
MANIFEST_PATH=vvs/corpus_manifest.csv
```

Where to get each key:

- **GROQ_API_KEY** — Go to console.groq.com → "API Keys" in the left sidebar → "Create API Key."
- **GEMINI_API_KEY** — Go to aistudio.google.com → click "Get API Key" (top right) → "Create API Key."
- **GCP_PROJECT_ID, VERTEX_INDEX_ID, VERTEX_INDEX_ENDPOINT_ID, VERTEX_DEPLOYED_INDEX_ID** — From the [gcloud](#) commands in Section 5.1.

5.4 setup.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running Day 5 Vector Search project setup..."

if [ ! -d "myenv" ]; then
    echo "Creating virtual environment 'myenv'..."
    python3 -m venv myenv
fi

echo "Activating myenv..."
source myenv/bin/activate

echo "Upgrading pip..."
pip install --upgrade pip

echo "Installing requirements..."
pip install -r requirements.txt

if [ ! -f ".env" ]; then
    echo "WARNING: .env file not found. Create one before running ingest.py –
    see guide.md Section 5.3."
else
    echo ".env file found."
fi

echo "Creating data/pdfs/ and logs/ directories (if not already
present)..."
mkdir -p data/pdfs
mkdir -p logs

echo "Touching output.log..."
touch output.log

if [ ! -f "README.md" ]; then
    echo "Creating empty README.md..."
    touch README.md
else
    echo "README.md already exists – not overwriting."
fi
```

```
echo ""  
echo "Setup complete. Edit .env before running ingest.py"
```

5.5 Execution Commands

```
# Step 1: Run ingestion (downloads PDFs, embeds, indexes)  
source myenv/bin/activate  
python vvs/ingest.py  
  
# Step 2: Run a basic query  
python vvs/query.py --question "What are the key components of a  
transformer architecture?"  
  
# Step 3: Run a metadata-filtered query  
python vvs/query.py --question "How is RLHF used in language model  
alignment?" --year 2023  
  
# Step 4: Run without synthesis (raw chunks only)  
python vvs/query.py --question "Attention mechanism variants" --top_k 10 --  
no-synthesis  
  
# Step 5: Verify output.log is being written  
tail -f output.log
```

5.6 Connecting Groq

Since prior GCP/Vertex AI setup is assumed already done, only the Groq-specific delta steps are needed:

1. **Verify `GROQ_API_KEY` is set in `.env`** — open `.env` and confirm the value is non-empty and not still the placeholder text.
2. **Confirm the `groq` package is installed:**

```
pip show groq
```

If it's missing, re-run `pip install -r requirements.txt`.

3. **Test connectivity:**

```
python -c "from groq import Groq; import os; from dotenv import  
load_dotenv; load_dotenv(); c=Groq(api_key=os.getenv('GROQ_API_KEY'));  
r=c.chat.completions.create(model='llama-3.3-70b-versatile', messages=  
[{'role': 'user', 'content': 'ping'}], max_tokens=5); print('Groq  
connected:', r.choices[0].message.content)"
```

A successful run prints `Groq connected: <short model reply>`.

5.7 vvs/setup.md Content

This is the full content that should be saved as `vvs/setup.md` in the project:

Vertex AI Vector Search – Configuration Reference

This file documents the live configuration of this project's Vertex AI resources, separately from `.env`, so the configuration is readable without needing to expose or load the actual environment file.

Index Configuration

- ****Display name:**** arxiv-ml-index
- ****Dimensions:**** 768
- ****Distance metric:**** DOT_PRODUCT_DISTANCE
- ****Algorithm:**** Tree-AH (treeAhConfig)
- ****Shard size:**** SHARD_SIZE_SMALL
- ****Approximate neighbors count:**** 150

Endpoint Configuration

- ****Display name:**** arxiv-ml-endpoint
- ****Deployed index ID:**** arxiv_ml_deployed_v1
- ****Public endpoint:**** enabled

Corpus Status

- ****Number of documents indexed:**** {FILL_AFTER_INGEST}
- ****Number of chunks indexed:**** {FILL_AFTER_INGEST}
- ****Last ingestion run date:**** {FILL_AFTER_INGEST}

Deploy Commands Reference

See *guide.md* Section 5.1 for the full `gcloud` commands used to create and deploy this index. In short:

```
gcloud ai indexes create --display-name="arxiv-ml-index" --metadata-
file=index_metadata.json ...
gcloud ai index-endpoints create --display-name="arxiv-ml-endpoint" ...
gcloud ai index-endpoints deploy-index ... --deployed-index-
id="arxiv_ml_deployed_v1" ...
```

Re-indexing: Drop-and-Recreate vs Upsert

- ****Upsert (default, used by ``ingest.py``):**** Re-running ``ingest.py`` against an already-populated index is safe for **new** documents – already-downloaded PDFs are skipped by filename check, and ``upsert_datapoints()`` is idempotent per ``chunk_id`` (re-upserting the same `chunk_id` with the same vector simply overwrites it). Use this for incremental corpus growth.
- ****Drop-and-recreate:**** Necessary if you change ``CHUNK_SIZE_TOKENS``, ``CHUNK_OVERLAP_TOKENS``, or the embedding model – these changes alter the

`chunk_id` scheme and/or the embedding space, making old and new datapoints incompatible within the same index. To drop and recreate:

1. ``gcloud ai index-endpoints undeploy-index`` to remove the old deployed index
2. ``gcloud ai indexes delete`` to delete the old Index resource
3. Re-run Section 5.1 Steps 3–5 to create a fresh Index and IndexEndpoint
4. Clear ``vvs/corpus_manifest.csv`` back to just its header row
5. Re-run ``vvs/ingest.py`` from scratch

SECTION 6 — `filtered_queries.md` Population Guide

After running `ingest.py` successfully, populate `vvs/filtered_queries.md` by running each query below and recording its results. This file becomes a living test log and a sanity-check artifact for verifying retrieval quality across different filter combinations.

For each query, run the CLI command, then fill in the markdown template underneath it with the actual top-5 chunk results from your terminal output.

Query 1 — Unfiltered semantic query

```
python vvs/query.py --question "What is the role of self-attention in transformer models?"
```

Filters applied: none

Result Log – Query 1

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

Query 2 — Year-filtered query (2023 only)

```
python vvs/query.py --question "What advances were made in instruction tuning?" --year 2023
```

Filters applied: `year=2023`

Result Log – Query 2

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

Query 3 — Doc-type filtered query

```
python vvs/query.py --question "How are positional encodings computed?" --
doc_type arxiv_paper
```

Filters applied: doc_type=arxiv_paper

Result Log – Query 3

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

Query 4 — Multi-concept query spanning multiple papers

```
python vvs/query.py --question "Compare RLHF and direct preference
optimization for alignment" --top_k 8
```

Filters applied: none (top_k increased to 8 to span more candidate papers)

Result Log – Query 4

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				
6				

	7									
	8									

Query 5 — Narrow technical term query

```
python vvs/query.py --question "rotary positional embedding RoPE"
```

Filters applied: none

#### Result Log – Query 5										
	Rank		chunk_id		distance		title		year	
	-----		-----		-----		-----		-----	
	1									
	2									
	3									
	4									
	5									

Query 6 — Year-filtered, earlier year (tests sparse-result behavior)

```
python vvs/query.py --question "early transformer architectures before attention" --year 2017
```

Filters applied: year=2017

#### Result Log – Query 6										
	Rank		chunk_id		distance		title		year	
	-----		-----		-----		-----		-----	
	1									
	2									
	3									
	4									
	5									

Query 7 — --no-synthesis raw-chunk-only query

```
python vvs/query.py --question "mixture of experts routing strategies" --no-synthesis
```

Filters applied: none; synthesis skipped

Result Log – Query 7

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

Query 8 — Combined year + doc_type filter

```
python vvs/query.py --question "scaling laws for compute-optimal training"
--year 2022 --doc_type arxiv_paper
```

Filters applied: year=2022, doc_type=arxiv_paper

Result Log – Query 8

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

Query 9 — Out-of-corpus probe (tests grounded-refusal behavior)

```
python vvs/query.py --question "What is the capital of France?"
```

Filters applied: none. This query is deliberately unrelated to the ML corpus — expected synthesis behavior is the fallback line, "I cannot find this in the indexed documents."

Result Log – Query 9

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				

****Synthesis fallback triggered?**** (yes/no):

Query 10 — High-top_k broad survey query

```
python vvs/query.py --question "What evaluation benchmarks are commonly
used for large language models?" --top_k 10
```

Filters applied: none; top_k=10

Result Log — Query 10

Rank	chunk_id	distance	title	year
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

SECTION 7 — questions.md

Project Evaluation & Code Review — Day 5: Vector Search & Embeddings

Q1: What is an embedding, and why do semantically similar texts produce similar embedding vectors?

****Answer:****

Write your answer here...

Q2: Why did this project choose 512-token chunks, and what problem does the 64-token overlap between consecutive chunks solve?

****Answer:****

Write your answer here...

Q3: What is the difference between the RETRIEVAL_DOCUMENT and RETRIEVAL_QUERY task types in the Gemini embedding API, and why does this project use a different one for indexing versus querying?

****Answer:****

Write your answer here...

Q4: Why does DOT_PRODUCT_DISTANCE produce the same ranking as cosine similarity for this project's embeddings? Show the math.

****Answer:****

Write your answer here...

Q5: What does the Vertex AI Index resource represent, what does the IndexEndpoint resource represent, and why does Vertex AI require both rather than just one?

****Answer:****

Write your answer here...

Q6: How does the Namespace `restricts` filter mechanism work inside Vertex AI Vector Search at query time, and why is filtering done this way instead of retrieving everything and filtering afterward?

****Answer:****

Write your answer here...

Q7: What are the tradeoffs between Approximate Nearest Neighbor (ANN) search and exact K-Nearest-Neighbors (KNN)? Describe a scenario where you would choose exact KNN despite its higher cost.

****Answer:****

Write your answer here...

Q8: Why does using the same embedding task_type for both indexing and querying matter for the dot-product score distribution returned by the index?

****Answer:****

Write your answer here...

Q9: Suppose Google releases a new version of the Gemini embedding model after you've already indexed 200 papers with the old version. How would you detect that embedding drift has occurred, and how would you handle migrating the index?

****Answer:****

Write your answer here...

Q10: Design a re-ranking layer to sit between Vertex AI's retrieval step and Groq's synthesis step. What signals would you use to re-rank the initial top-K candidates, and why does high retrieval recall not guarantee high final answer quality?

****Answer:****

Write your answer here...

Final Implementation Checklist

- ☒ No placeholders, no `# TODO`, no `...` ellipsis anywhere in code — every function above has a complete body.
- ☒ Every `import` statement is present in every file shown.
- ☒ All error handling uses `try/except` with specific exception types (`PdfReadError`, `ResourceExhausted`, `NotFound`, `groq.APIError`, `groq.RateLimitError`, `OSError`/`IOError`) — no bare `except:` anywhere.
- ☒ Every module calls `get_logger()` from `utils/logger.py` — no `print()` calls inside `src/`, `config/`, or `utils/` (the two `vvs/` CLI scripts use `print()` only for the user-facing terminal)

output that is explicitly part of their CLI contract — query results and the synthesized answer — while all diagnostic/audit logging still goes through `get_logger()`.

- ☒ `output.log` captures API calls, retries, batch progress, errors, and the final ingestion summary.
- ☒ All shell commands are macOS bash-compatible.
- ☒ Virtual environment is named `myenv` throughout.
- ☒ `corpus_manifest.csv` is written in append mode, one row per successfully processed document, so a crash mid-run does not lose already-completed progress.
- ☒ All assumptions are stated at the top of this document, before Section 1.

One architectural note carried forward from Section 4.6: the manifest schema specified in Section 2 is document-level only (`doc_id, title, year, doc_type, num_pages, num_chunks`). This means `query.py`'s metadata hydration step can recreate a chunk's parent-document title/year/doc_type, but not its original chunk text or precise page number, since neither is persisted anywhere outside the in-memory chunk objects that exist transiently during `ingest.py`'s run. This is implemented honestly above (the gap is visible in `hydrate_chunk_metadata()`'s output and called out explicitly) rather than papered over. If you want full chunk text available at query time for synthesis, the cleanest extension is a chunk-level side-store (SQLite, JSON, or a second CSV keyed by `chunk_id`) written during `upsert_chunks()` in `ingest.py`.